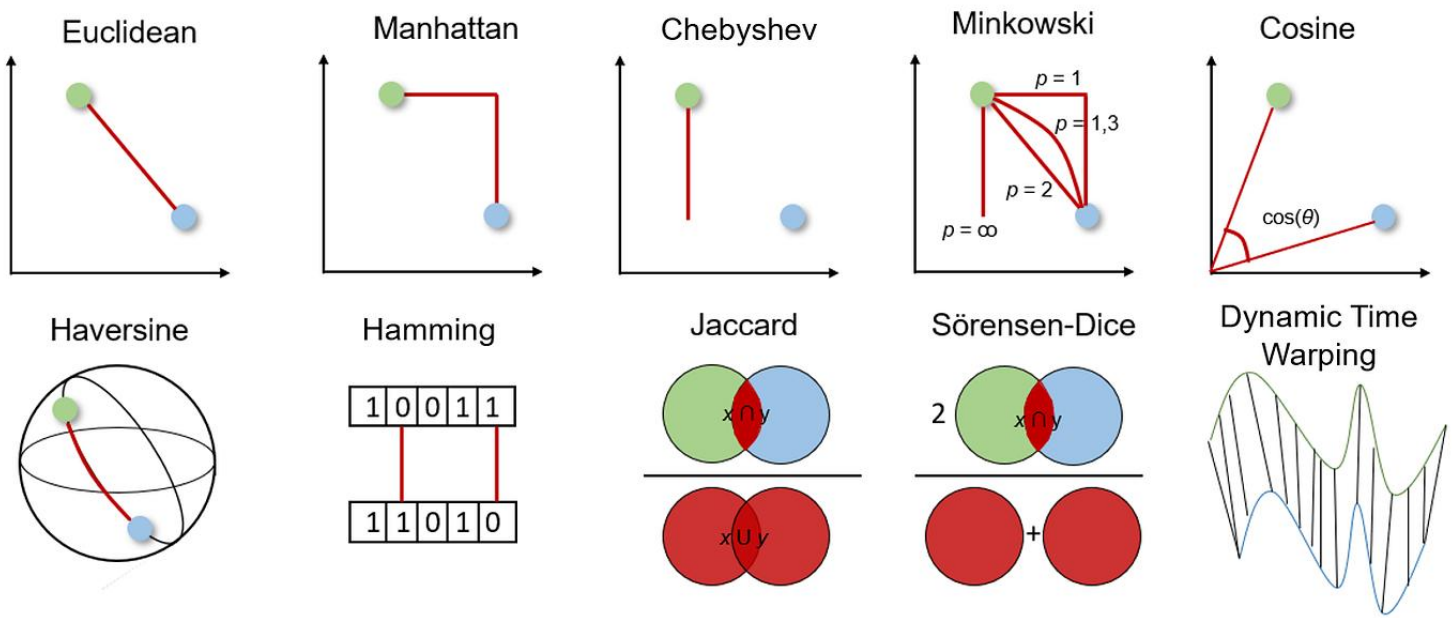
香港教育大學
The Education University of Hong Kong

MSc(AIET) 2025-26

MTH6184 Data Mining in STEM Education

Chapter 6. kNN and Naïve Bayes

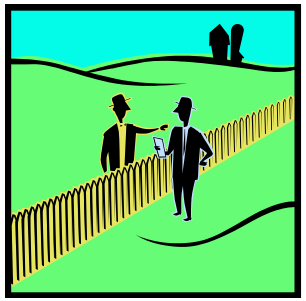
Instructor: Philip Yu

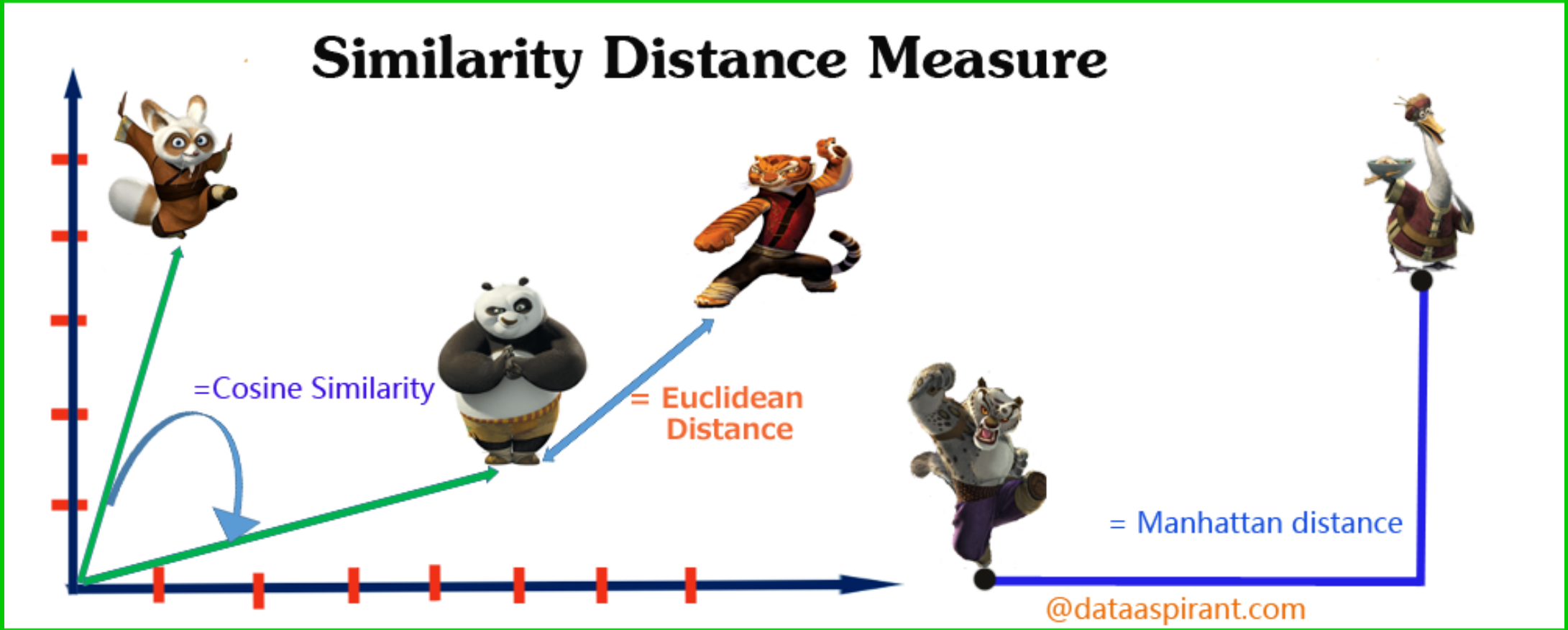


Chapter Outcomes

After completing this chapter, you are able to:

- Understand what a K-Nearest Neighbors (KNN) is
  - Distance Measures
  - Weighted Voting / Average
  - Practical Issues of Implementing KNN
  - Pros and Cons of KNN
- Understand what a Naïve Bayes Classifier is
  - Pros and Cons of Naïve Bayes Classifier

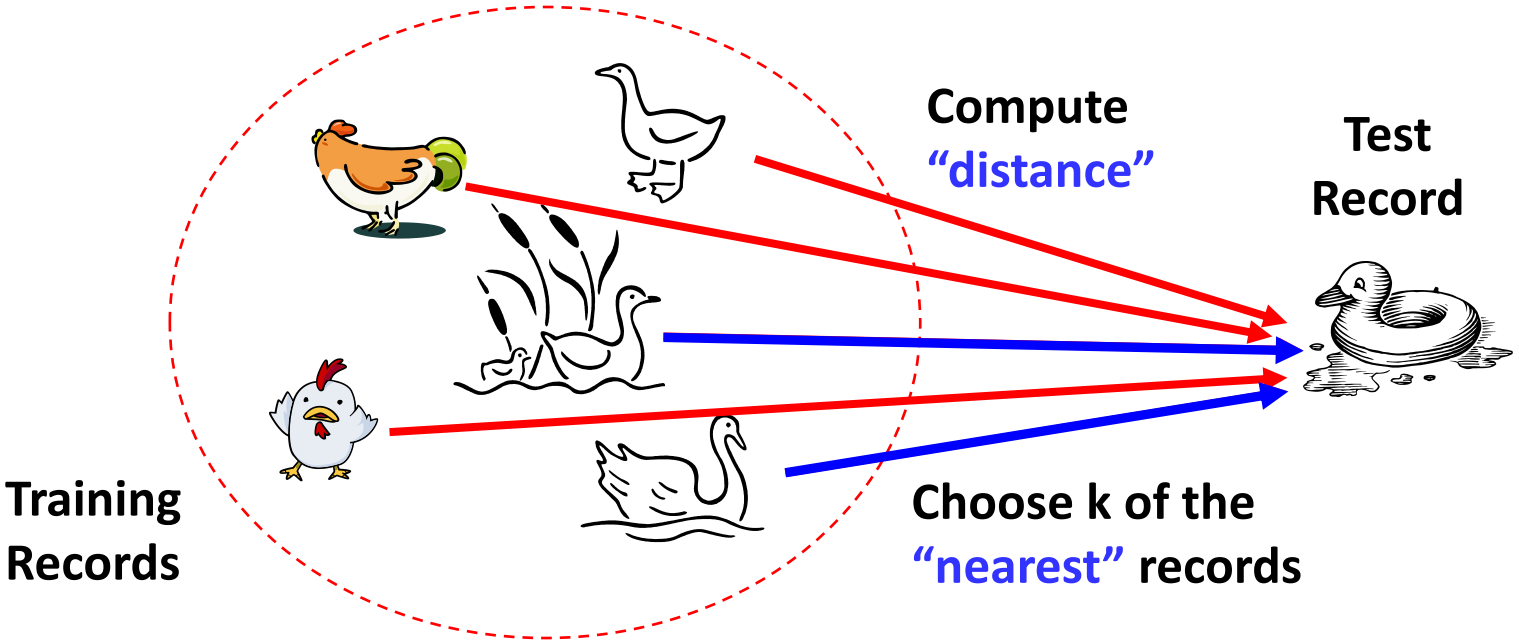




KNN Classifier and Regression

K-Nearest Neighbors (KNN)

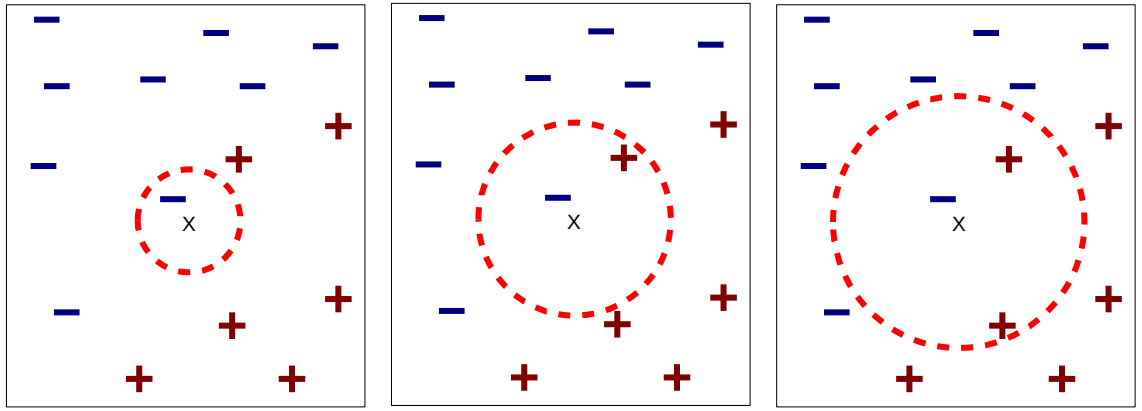
- KNN is a supervised learning algorithm used for classification and regression.
- **Basic idea:** If it walks like a duck, quacks like a duck, then it's probably a duck



- People use nearest neighbor methods all the time
  - Diagnosing a disease
  - Guessing the prices charged in a restaurant

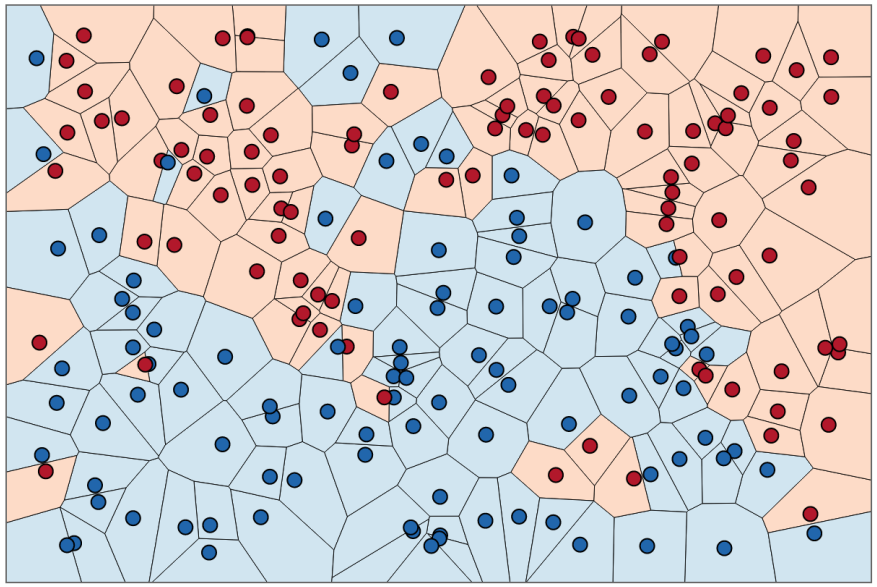
Definition of Nearest Neighbor

k-nearest neighbors of a record x are data points that have the k smallest distance to x

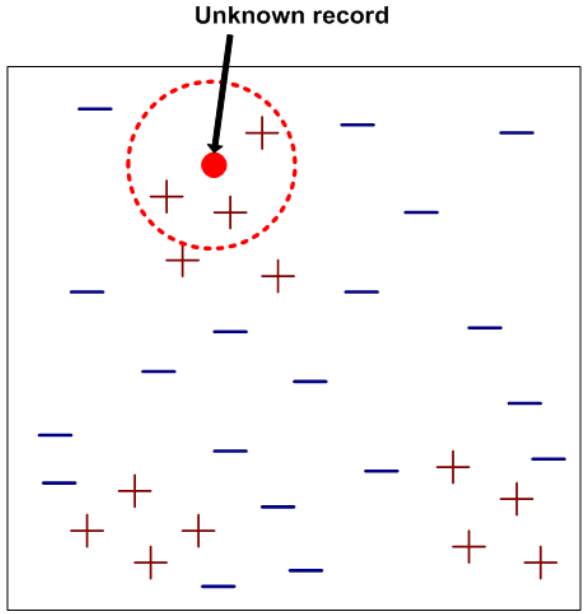


(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

Voronoi cell visualization of nearest neighborhoods (i.e., 1NN)



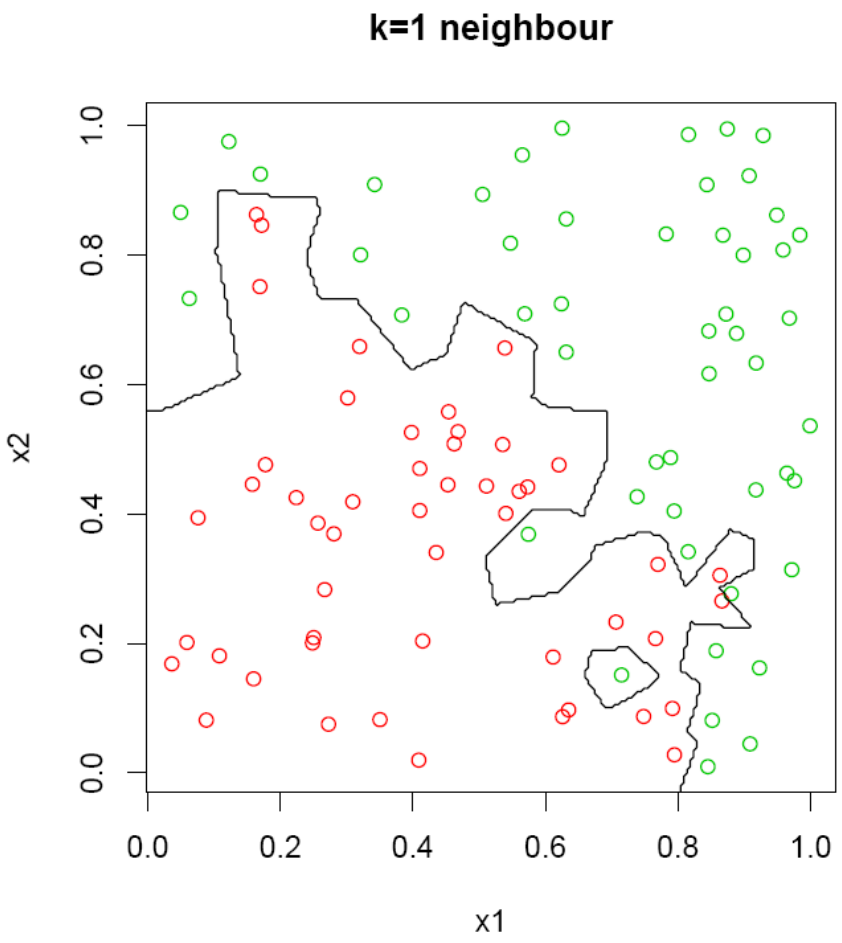
K-Nearest Neighbors Algorithm

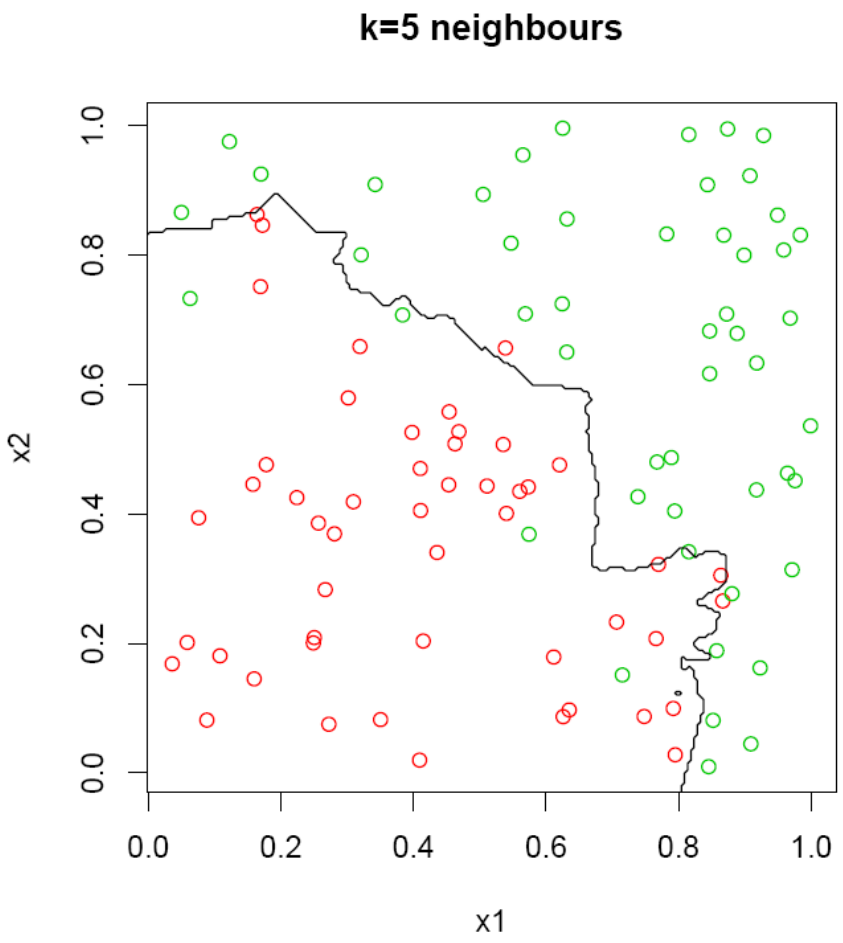


- Requires three things:
  1. The set of training records
  2. Distance measure to compute distance between records, eg. Euclidean distance, see
  3. The value of *k*, the number of nearest neighbors to retrieve
- To classify/predict an unknown record:
  - Compute distance to every training record
  - Identify *k* nearest neighbors
  - Combine the target values of these nearest neighbors to predict the target of unknown record (e.g., by taking majority vote or simple average (weights = ‘uniform’))
  - Weighted voting/mean: Sometimes, it is better to weight the neighbors such that nearer neighbors contribute more to the fit. (weights = ‘distance’ assigns weights proportional to the inverse of the distance from the query record)

```
sklearn.neighbors.KNeighborsClassifier / KNeighborsRegressor:
n_neighbors = 5,
Weights=‘uniform’ | ‘distance’,
Metric = ‘minkowski’ | ‘euclidean’ | ’manhattan’ | ’cosine’,
p = 2
```

Example 1: 1-NN and 5-NN



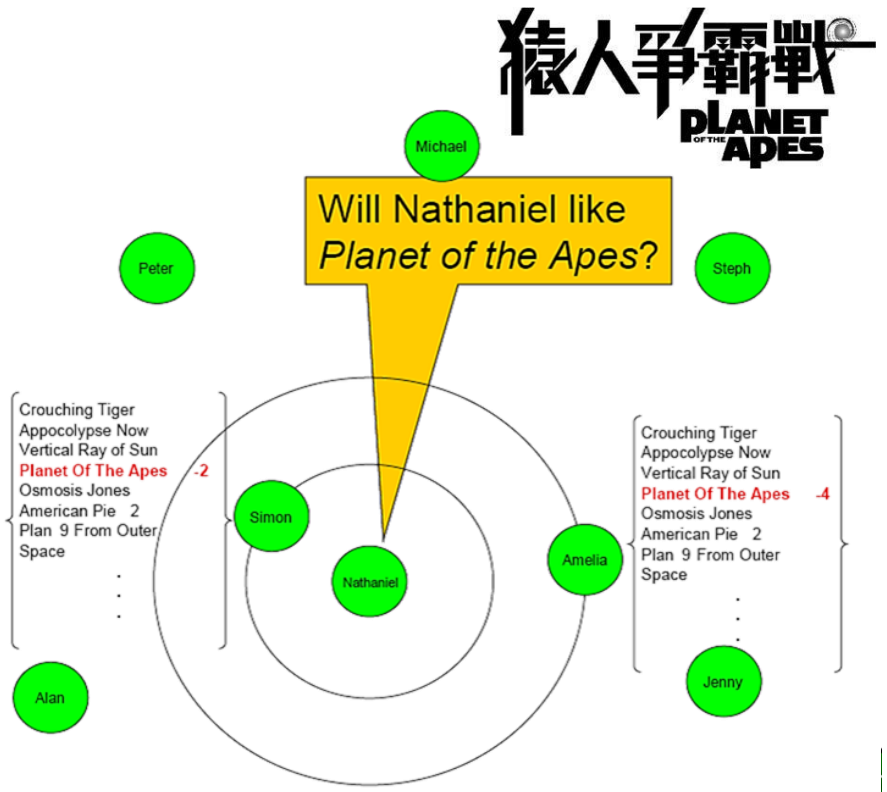


Applications of KNN

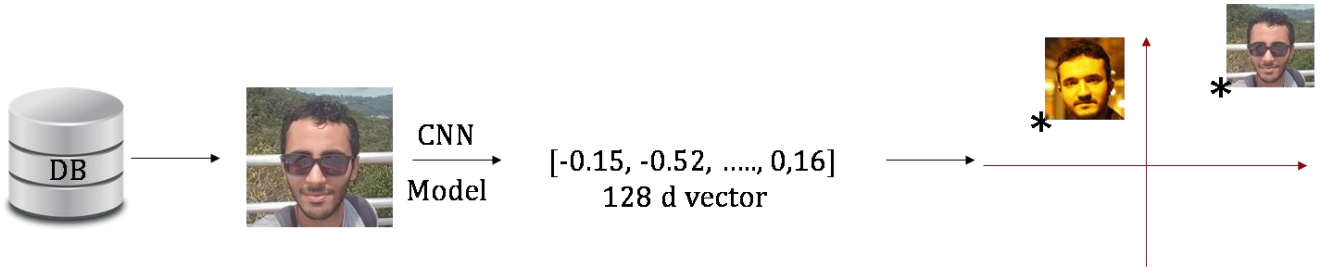
- **Recommender system** (Amazon, Netflix, etc)
  - What rating would Nathaniel give to Planet of the Apes?
    - Simon, distance 1, rated it -2 (below his mean)
    - Amelia, distance 2, rated it -4 (below her mean)
  - Using weighted average inverse to distance, it is predicted that Nathaniel would rate it

$$\frac{\frac{1}{1}(-2)+\frac{1}{2}(-4)}{\frac{1}{1}+\frac{1}{2}} = -2.667 \text{ (below his mean)}$$

  - Also called user-based collaborative filtering



- **Face recognition**



The face is fully described with the 128 d vector

See https://devmesh.intel.com/projects/face-recognition-using-knn

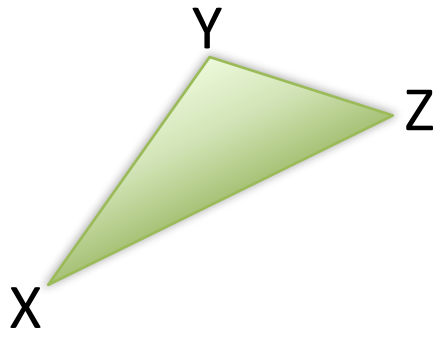
What Makes a Good Distance Measure?

A good distance (or dis-similarity) measure:

- Non-negativity: $d(X, Y) \geq 0$
  - Distance is always non-negative
- Symmetry: $d(X, Y) = d(Y, X)$
  - Distance from "X to Y" is distance from "Y to X"
- Identity: $d(X, Y) = 0$ if and only if $X = Y$

Other conditions:

- Triangle inequality: $d(X, Z) \leq d(X, Y) + d(Y, Z)$
  - Metric requirement: distance from "X to Z" must be less than or equal to distance from "X to Y to Z"



Some Good Distance Measures

- Given two records with L continuous feature variables:

$$\boldsymbol{x} = (x_1, \dots, x_L)'$$
$$\boldsymbol{y} = (y_1, \dots, y_L)'$$

the Minkowski distance between X and Y is:

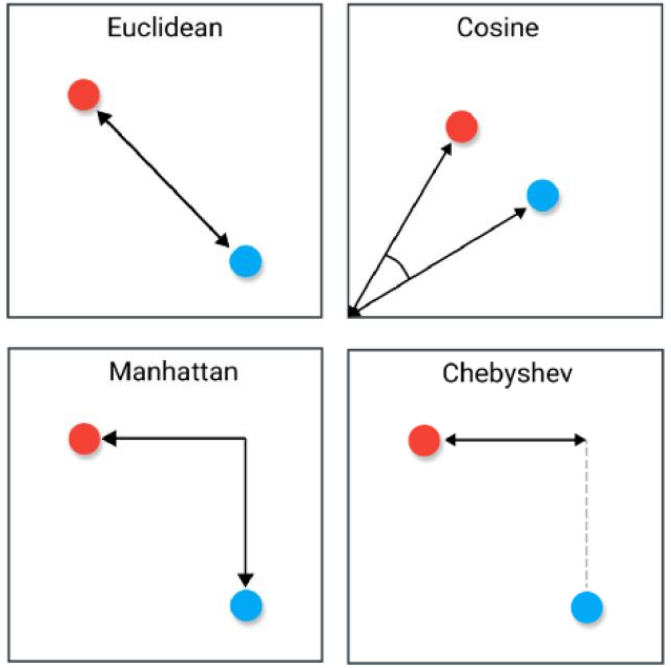
$$d(\boldsymbol{x}, \boldsymbol{y}) = (|x_1 - y_1|^p + \cdots + |x_L - y_L|^p)^{\frac{1}{p}}$$

- Euclidean distance: $p = 2$ (most widely used)
- Manhattan (City-block) distance: $p = 1$
- Chebyshev distance: $p = \infty$

$$d(\boldsymbol{x}, \boldsymbol{y}) = \max\{|x_1 - y_1|, \dots, |x_L - y_L|\}$$

- Cosine Distance: $d(\boldsymbol{x}, \boldsymbol{y}) = 1 -$ Cosine_Similarity

where Cosine_Similarity $= \frac{\boldsymbol{x}'\boldsymbol{y}}{\sqrt{\boldsymbol{x}'\boldsymbol{x}}\sqrt{\boldsymbol{y}'\boldsymbol{y}}}$ is cosine angle between of $\boldsymbol{x}$ and $\boldsymbol{y}$



- **Q:** Let $r(\boldsymbol{x}, \boldsymbol{y}) = corr(\boldsymbol{x}, \boldsymbol{y})$. Is $d = 1 - r^2$ a good measure?

Example 2

- Student X got 30 marks in a test and 12 marks in a quiz.
- Student Y got 40 marks in the test and 8 marks in the quiz.
- What is the Euclidean distance between these two students?

$$d(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{(30-40)^2 + (12-8)^2} = 10.77$$

- When measuring distance, input variables with very large values (relative to the other input variables) will overwhelm the distance calculation.
  - Eg., income has a large range 30,000-100,000, whereas years_of_service takes on values 0-20
- To avoid this situation we can use scaling.
  - Suppose the test has a full marks of 50 and the quiz has a full marks of 20. We may convert them by dividing the test score by 50 and the quiz score by 20.
- Common scaling methods: min-max normalization or z-score standardization

Example 3

- Person A = 50-year-old male
- Person B = 20-year-old male
- Person C = 50-year-old female
- Suppose that the Age variable has a range = 50, minimum = 10, mean = 45, and standard deviation = 15

Person	Age	Age_{Norm}	Age_{Std}	Gender
A	50	$\frac{50-10}{50} = 0.8$	$\frac{50-45}{15} = 0.33$	Male (0)
B	20	$\frac{20-10}{50} = 0.2$	$\frac{20-45}{15} = -1.67$	Male (0)
C	50	$\frac{50-10}{50} = 0.8$	$\frac{50-45}{15} = 0.33$	Female (1)

- Which person, B or C, is closer to Person A?

- Age unnormalized

$$d(A,B) = \sqrt{(50-20)^2 + 0^2} = 30$$
$$d(A,C) = \sqrt{(50-50)^2 + 1^2} = 1$$

  - Therefore Person C is closer to Person A
  - Is Person B really 30 times more distant than Person C is to Person A?
  - Perhaps neglecting to normalize the values of Age is creating this discrepancy.
- Age Normalized using Min-Max Normalization
  - Values lie in the range [0, 1]

$$d_{MMN}(A,B) = \sqrt{(0.8-0.2)^2 + 0^2} = 0.6$$
$$d_{MMN}(A,C) = \sqrt{(0.8-0.8)^2 + 1^2} = 1.0$$

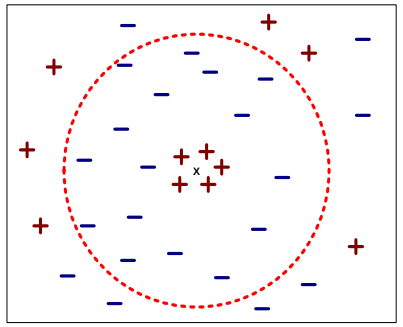
  - In this case, Person B is now closer to Person A
- Age Standardized using Z-Score Standardization
  - most values are typically in the range [-3, 3]

$$d_{Zscore}(A,B) = \sqrt{(0.33-(-1.67))^2 + 0^2} = 2.0$$
$$d_{Zscore}(A,C) = \sqrt{(0.33-0.33)^2 + 1^2} = 1.0$$

  - Now, Person C is closer to Person A
- Which Method should I choose?

Weighted Voting/Average

- Once the *K* nearest neighboring records are identified, the next step is to combine their targets to provide a classification / prediction.



- Simple Unweighted Voting/Average
  - Take the majority vote of class labels (simple average of the target values) among the *k* nearest neighbors
- Weighted Voting/Average
  - Weigh the votes (target values) according to their distances
    - The closer the neighbor, the more influence it has in the classification/prediction decision
  - This method assumes a closer neighbor is more similar, and therefore its vote should be weighted more heavily, as compared that of more distant neighbors
  - A “tie” is unlikely to occur using this approach

Example 4: Weighted Voting

- Suppose a student got 33 points for the exercise and 12.5 hours of study
- Target1 = Test (Fail, Satisfactory, Good)
- Its k=3 nearest neighbors from last year are:

Student	Target1	Ex	Study	Ex_{MMN}	$Study_{MMN}$
New	?	33	12.5	0.05	0.25
A	Fail	32.8	12.4	0.0467	0.2471
B	Satisfactory	33.2	10.5	0.0533	0.1912
C	Satisfactory	35.5	13.5	0.0917	0.2794

- Majority vote: predicted test result is Satisfactory
- The distance of students A, B, and C to this new student are:

$$d(new, A) = \sqrt{(.05 - .0467)^2 + (.25 - .2471)^2} = .004393$$

$$d(new, B) = \sqrt{(.05 - .0533)^2 + (.25 - .1912)^2} = .058893$$

$$d(new, C) = \sqrt{(.05 - .0917)^2 + (.25 - .2794)^2} = .051022$$

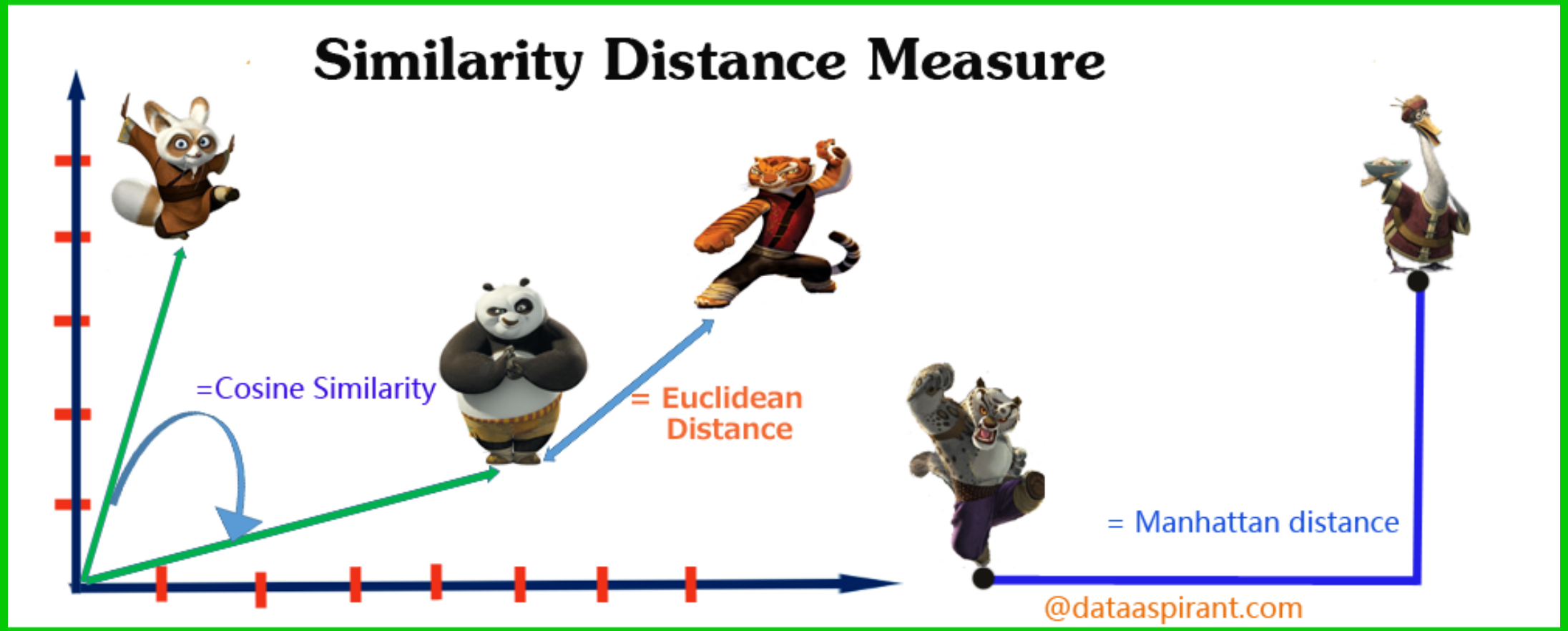
- Votes of these students are weighted according to the reciprocal of their distances to the new student:
  - Vote for classifying the new student as Fail: $Vote(Fail) = \frac{1}{d(new,A)} = \frac{1}{.004393} = 227.6$
  - Vote for classifying the new student as Sat: $Vote(Sat) = \frac{1}{.058893} + \frac{1}{.051022} = 36.6$
  - Vote for classifying the new student as Good: $Vote(Good) = 0$

Example 5: Weighted Average

- Target2 (y) = Exam mark

Student	Target2	Ex	Study	Ex_{MMN}	$Study_{MMN}$	Distance
New	?	33	12.5	0.05	0.25	--
A	60	32.8	12.4	0.0467	0.2471	0.004393
B	61	33.2	10.5	0.0533	0.1912	0.058893
C	65	35.5	13.5	0.0917	0.2794	0.051022

- Weights = Uniform: $\hat{y}_{new} = \frac{\sum_i y_i}{3} = \frac{60+61+65}{3} = 62$
- Weights = Distance: $\hat{y}_{new} = \frac{\sum_i w_i y_i}{\sum_i w_i} = \frac{\frac{60}{.004393}+\frac{61}{.058893}+\frac{65}{.051022}}{\frac{1}{.004393}+\frac{1}{.058893}+\frac{1}{.051022}} = 60.4$
- Since student A is the closest to the new student, its target value of 60 marks makes a significant contribution to the predicted target exam mark.



KNN: Practical Issues

Variable Relevance / Selection

- Not all input variables may be relevant to classification and regression
  - Variable selection is needed before applying kNN
  - Especially for high dimensional data (curse of dimensionality)

1	1	1	1	1	1	1	1	0

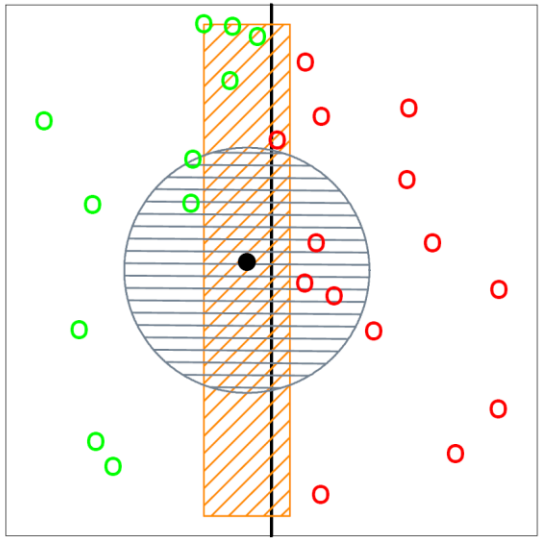
d = 1.41

0	1	1	1	1	1	1	1	1

d = 1.46

2.4	1.2	.8	.9	1.2	1	.9	.8	0

5-Nearest Neighborhoods



- The **vertical line** separates class red and green perfectly.
- The **vertical strip** denotes the 5-nearest-neighbor region using only the horizontal axis to find the nearest neighbors for the target point (solid black dot).
- The **circle** shows the 5-nearest-neighbor region using both axes, and we see in this case it has extended into the class-red region (and is dominated by the wrong class in this instance).

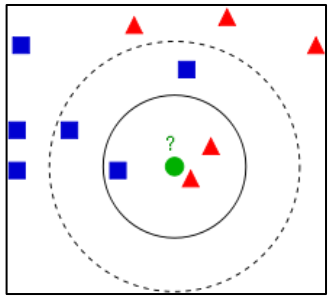
Variable Relevance / Selection

- Solution 1: Stretching the axes
  - Assign more weights to the more important feature variables (i.e., weighted distance)

$$d(\boldsymbol{x}, \boldsymbol{y}) = (W_1|x_1 - y_1|^p + \cdots + W_L|x_L - y_L|^p)^{\frac{1}{p}}$$

  - Eg. Each feature variable is weighted by the absolute value of its correlation to the target variable or assigned by domain experts
- Solution 2: Use PCA or other dimension reduction methods
  - Aim to reduce the number of dimensions but at the same time to keep as much information as possible
  - Select the PCs and standardize each PC by z-score before applying KNN (Why?)
- Remarks:
  - How to calculate distance between two records having a mix of categorical and continuous input variables? Use Gower distance (see https://github.com/wwwjk366/gower)
  - KNN does not allow missing input values. Impute first!
  - Use a balanced training dataset so as to include a sufficient number of records with common, as well as less-common, target classes (Oversampling for less frequent classes)

Choosing K (hyperparameter)

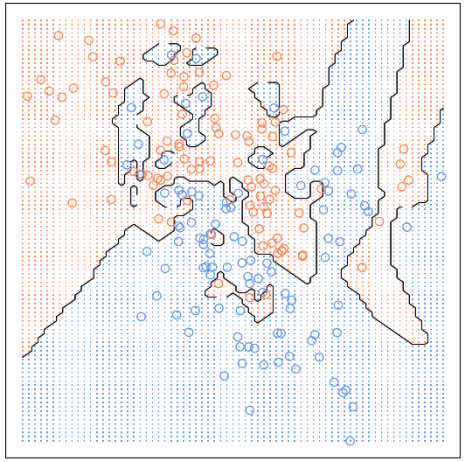


- Smaller K
  - A smaller value of K may provide more flexible (but unstable) fit, which will have lower bias but higher variance (due to outliers).
- Larger K
  - A larger value of K may have smoother decision boundaries (but ignore local pattern) which means lower variance but increased bias.
- Solutions:
  - Using cross-validation to determine the optimal value of K which minimizes the classification error / mean square error on the validation data
  - A rule of thumb: for training data of size n,

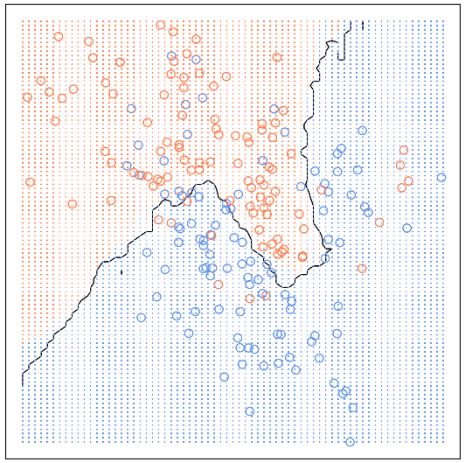
$$K \leq \sqrt{n}$$

- Algorithmic issue: speeding up kNN search
  - Naïve search takes time $O(n^2)$ for training set of size n. Slow!
  - Luckily there are data structures for speeding up nearest neighbor search, like: Ball trees or KD trees

1-nearest neighbours



20-nearest neighbours



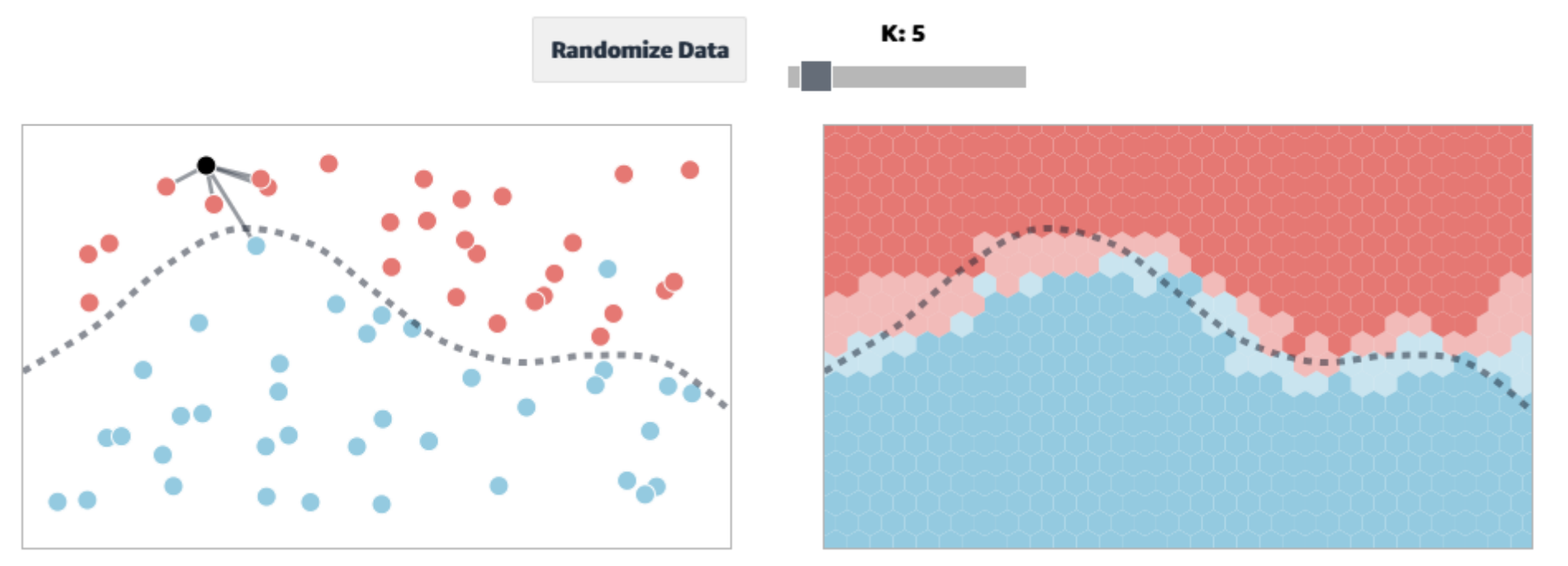
Pros and Cons of KNN

- Pros:
  - KNN is simple to understand and easy to implement.
  - Efficient method for small datasets with less number of features.
  - Basically zero to little training time. The training dataset is stored and is used only when making predictions therefore making it more quick than Regression.
- Cons:
  - Large datasets take longer to process.
  - Requires feature scaling, and inability to do will result in wrong predictions.
  - KNN can suffer from skewed class distributions (or imbalanced dataset). For example, if a certain class is very frequent in the training set, it will tend to dominate the majority voting of the new example.
  - Sensitive to outliers
  - the accuracy of KNN can be severely degraded with high-dimensional data because there is little difference between the nearest and farthest neighbor.

Class Activity 1:

KNN

MLU-EXPLAIN



https://www.analyticsvidhya.com/blog/2021/09/naive-bayes-algorithm-a-complete-guide-for-data-science-enthusiasts/

Naïve Bayes Classifier

Bayes' Theorem

- A probabilistic framework for solving classification problems
- Conditional Probability:

$$P(Y|X) = \frac{P(X,Y)}{P(X)} \qquad P(X|Y) = \frac{P(X,Y)}{P(Y)}$$

- Bayes' theorem:

$$P(Y|X) = \frac{P(X|Y)\,P(Y)}{P(X)} = \frac{P(X|Y)\,P(Y)}{\sum_y P(X|Y=y)\,P(Y=y)}$$

- Eg. Given the following information:
  - A doctor knows that meningitis(腦膜炎) causes stiff neck(頸硬) 50% of the time
  - Prior probability of any patient having meningitis is 1/50,000
  - Prior probability of any patient having stiff neck is 1/20
- If a patient has stiff neck, what's the probability he/she has meningitis?

$$P(M|S) = \frac{P(S|M)\,P(M)}{P(S)} = \frac{0.5 \times \frac{1}{50000}}{\frac{1}{20}} = 0.0002 = \frac{1}{5000}$$

Naïve Bayes Classifier

- Given a data set on a class label Y and inputs $X_1, \dots, X_k$, naive Bayes classifier aims to predict Y by using Maximum A Posteriori (MAP) estimation which maximizes $P(Y|X_1, \dots, X_k)$.
- Can we estimate $P(Y|X_1, \dots, X_k)$ directly from the data? Possibly not. Why?
- Use Bayes' Theorem: $P(Y|X_1, \dots, X_k) = \frac{P(X_1,\dots,X_k|Y)P(Y)}{P(X_1,\dots,X_k)}$
- Since $P(X_1, \dots, X_k)$ is constant, we can choose value of Y that maximizes $P(X_1, \dots, X_k|Y)P(Y)$.
- Naïve Bayes classifier assumes conditional independence:

$$P(X_1, \dots, X_k|Y) = P(X_1|Y) \cdots P(X_k|Y),$$

  we can estimate $P(X_i|Y)$ and $P(Y)$ directly from the data.
- New point is classified to class label Y=y if

$$\max_y P(Y = y) \prod_{i=1}^{k} P(X_i|Y = y)$$

Example 6

```
CategoricalNB is designed
for discrete features:
```

```
sklearn.naive_bayes.CategoricalNB:
alpha = 1.0
```

- Given the following symptoms and diagnosis for 8 patients:

chills	runny noise	headache	fever	flu?
Yes	No	Mild	Yes	No
Yes	Yes	No	No	Yes
Yes	No	Strong	Yes	Yes
No	Yes	Mild	Yes	Yes
No	No	No	No	No
No	Yes	Strong	Yes	Yes
No	Yes	Strong	No	No
Yes	Yes	Mild	Yes	Yes

- Will a patient with the following symptoms have the flu?

chills	runny noise	headache	fever	flu?
Yes	No	Mild	Yes	?

- Class: Y = flu?
  - P(Y=No) = 3/8 and P(Y=Yes) = 5/8
- For discrete X_i: CategoricalNB

$$P(X_i = x_k | Y = y_j) = N_{kj}/N_j$$

where N_{kj} is number of obs. having attribute $X_i = x_k$ and belonging to class $Y = y_j$

- Examples:

P(fever=Yes | Y=Yes) = 4/5
P(headache=Mild | Y=No) = 1/3

- If $N_{kj} = 0$, use smoothing:

$$P(X_i = x_k | Y = y_j) = \frac{N_{kj} + \alpha}{N_j + \alpha n_i}$$

where α (=1) is smoothing hyperparameter and n_i is the number of levels of X_i

Example 6: Naïve Bayes Classifier

- Given a test record:

  X = (chills = Yes, runny noise = No, headache = Mild, fever = Yes)

- Let $\alpha = 1$ (add-one smoothing)
- P(X|Y=Yes) = P(chills=Yes | Y=Yes) × P(runny noise = No | Y=Yes) × P(headache = Mild| Y=Yes) × P(fever = Yes| Y=Yes)

  $$= \frac{3+1}{5+2} \times \frac{1+1}{5+2} \times \frac{2+1}{5+3} \times \frac{4+1}{5+2} = 0.0437$$

- P(X|Y=No) = P(chills=Yes | Y=No) × P(runny noise = No | Y=No) × P(headache = Mild| Y=No) × P(fever = Yes| Y=No)

  $$= \frac{1+1}{3+2} \times \frac{2+1}{3+2} \times \frac{1+1}{3+3} \times \frac{1+1}{3+2} = 0.0320$$

- P(Y=Yes|X) ∝ P(X|Y=Yes) P(Y=Yes) = 0.0437 x 5/8 = 0.0273
- P(Y=No|X) ∝ P(X|Y=No) P(Y=No) = 0.0320 x 3/8 = 0.0120

  => Y = Yes

chills	runny noise	headache	fever	flu?
Yes	No	Mild	Yes	No
Yes	Yes	No	No	Yes
Yes	No	Strong	Yes	Yes
No	Yes	Mild	Yes	Yes
No	No	No	No	No
No	Yes	Strong	Yes	Yes
No	Yes	Strong	No	No
Yes	Yes	Mild	Yes	Yes

```
BernoulliNB is designed
for binary features:
```

```
sklearn.naive_bayes.BernoulliNB:
alpha = 1.0
```

Example 7: NB Classifier for Spam Filtering

	Doc	Documents	Class
Training	1	"send us your password"	spam
	2	"send us your review"	ham
	3	"password review"	ham
	4	"review us"	spam
	5	"send your password"	spam
	6	"send us your account"	spam
Test	7	"review us now"	?

Word, w	$P(w\|S)$	$P(w\|H)$
review	1/4	2/2
send	3/4	1/2
us	3/4	1/2
your	3/4	1/2
password	2/4	1/2
account	1/4	0/2

- **Priors:** $P(S) = \frac{4}{6}$ $P(H) = \frac{2}{6}$
- **Conditional Probabilities:** $P(w|c) = \frac{count(w,c)}{\sum_{w \in C} count(w,c)}$ (assume no smoothing)
- Assuming that the words in each message are independent events:

$$P(Doc7|S) = P(\{1,0,1,0,0,0\}|S) = \frac{1}{4}\left(1-\frac{3}{4}\right)\frac{3}{4}\left(1-\frac{3}{4}\right)\left(1-\frac{2}{4}\right)\left(1-\frac{1}{4}\right) = 0.0044$$

$$P(Doc7|H) = P(\{1,0,1,0,0,0\}|H) = \frac{2}{2}\left(1-\frac{1}{2}\right)\frac{1}{2}\left(1-\frac{1}{2}\right)\left(1-\frac{1}{2}\right)\left(1-\frac{0}{2}\right) = 0.0625$$

- **Choosing a class:** $P(S|Doc7) = \frac{P(Doc7|S)P(S)}{P(Doc7|S)P(S)+P(Doc7|H)P(H)} = \frac{0.0044\times\frac{4}{6}}{0.0044\times\frac{4}{6}+0.0625\times\frac{2}{6}} = 0.123$

$$P(H|Doc7) = 1 - 0.123 = 0.877 \checkmark$$

- Note that the independence assumption between words will not in general be satisfied, but it can be useful for a naive approach.

Conditional Probability for Continuous Input Variables

For continuous X_i:

- Use binning

- Use Gaussian probability density function (GaussianNB):

```
sklearn.naive_bayes.GaussianNB
```

  - Assume $X_i|Y = y_j \sim N(\mu_{ij}, \sigma_{ij}^2)$
  - Use training data to estimate μ_{ij} and σ_{ij}^2

$$
\begin{aligned}
P(X_i = x|Y) &\approx P(x \le X_i \le x + \epsilon|Y) \\
&= \int_x^{x+\epsilon} f(X|Y)dX \\
&\to \epsilon f(x|Y) \text{ when } \epsilon \to 0.
\end{aligned}
$$

$$
P(X_i|Y = y_j) \propto \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} \exp\left(-\frac{(X_i - \mu_{ij})^2}{2\sigma_{ij}^2}\right)
$$

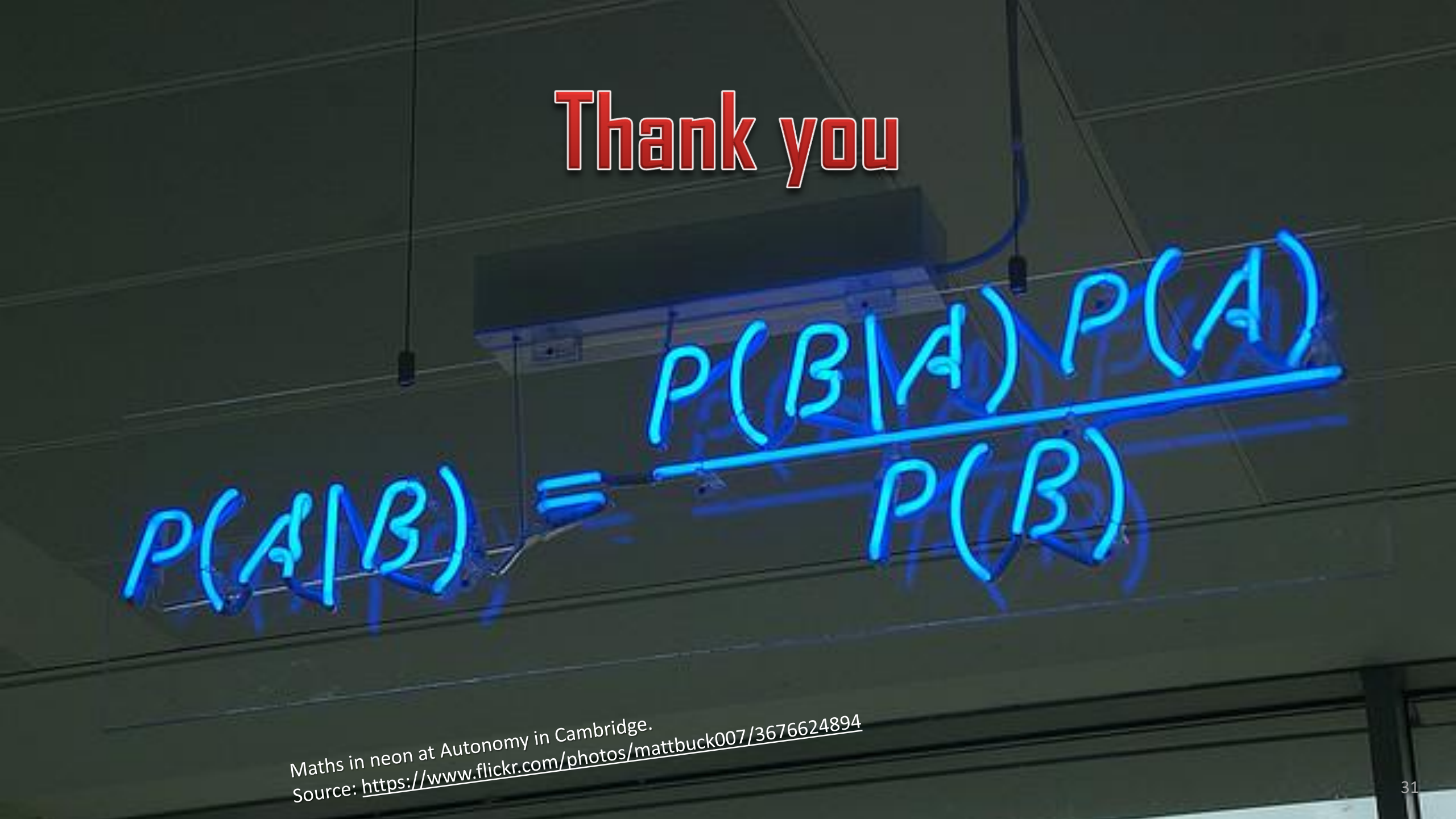
Class Activity 2:

Lab 6: KNN and Naive Bayes

Pros and Cons of Naive Bayes Classifier

- Pros:
  - Naive Bayes is a simple and easy to implement algorithm. Because of this, it might outperform more complex models for small data.
  - Naive Bayes works well with numerical and categorical features.
  - Robust to irrelevant features
  - Low computation costs for training and prediction
- Cons:
  - If you have no occurrences of a class label and a certain feature value together (e.g. class="spam", contains="$$$") then the frequency-based probability estimate will be zero. (Use smoothing)
  - Naive Bayes works well as long as the categories are kept simple. For instance, it works well for problems involving keywords as features (e.g. spam detection), but it might not work well when the relationship between words is important (e.g. sentiment analysis).
  - Independence assumption may not hold for some features (Use other models such as Bayesian Networks)

Thank you



Maths in neon at Autonomy in Cambridge.
Source: https://www.flickr.com/photos/mattbuck007/3676624894